

Exercise 2: Aggregates and Operators

Nolwandle Lala Dhladhla

Data Analytics

Bright learn

Date : 10 September 2026

1.The students Table

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
5 name STRING,  
6 age INT,  
7 department STRING  
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.students  
11 VALUES  
12 (1,'Alice',20,'IT'),  
13 (2,'Bob',22,'HR'),  
14 (3,'Charlie',21,'IT'),  
15 (4,'Diane',23,'Finance'),  
16 (5,'Eve',22,'HR');  
17  
18 SELECT*  
19 FROM brightlearn_course_students.e2.students;  
20  
21
```

Below the query editor, the results are displayed as a table with 5 rows and 4 columns: student_id, name, age, and department.

student_id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diane	23	Finance
5	Eve	22	HR

The interface also shows a sidebar with navigation options and a top bar with search and user information.

The screenshot shows the Databricks SQL Editor interface with a full SQL script. The script includes the following lines:

```
1 CREATE CATALOG IF NOT EXISTS brightlearn_course_students;  
2 CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.e2;  
3 CREATE TABLE IF NOT EXISTS brightlearn_course_students.e2.students(  
4   student_id INT,  
5   name STRING,  
6   age INT,  
7   department STRING  
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.students  
11 VALUES  
12 (1,'Alice',20,'IT'),  
13 (2,'Bob',22,'HR'),  
14 (3,'Charlie',21,'IT'),  
15 (4,'Diane',23,'Finance'),  
16 (5,'Eve',22,'HR');  
17  
18 SELECT*  
19 FROM brightlearn_course_students.e2.students;  
20  
21
```

The interface also shows a sidebar with navigation options and a top bar with search and user information.

List all distinct departments in the students table.

The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
6 age INT,  
7 department STRING  
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.students  
11 VALUES  
12 (1,'Alice',20,'IT'),  
13 (2,'Bob',22,'HR'),  
14 (3,'Charlie',21,'IT'),  
15 (4,'Diane',23,'Finance'),  
16 (5,'Eve',22,'HR');  
17  
18 SELECT  
19 department  
20 FROM brightlearn_course_students.e2.students;  
21  
22
```

Below the query, the results are displayed in a table format:

department
IT
HR
IT
Finance
HR

Get the average age of students per department.

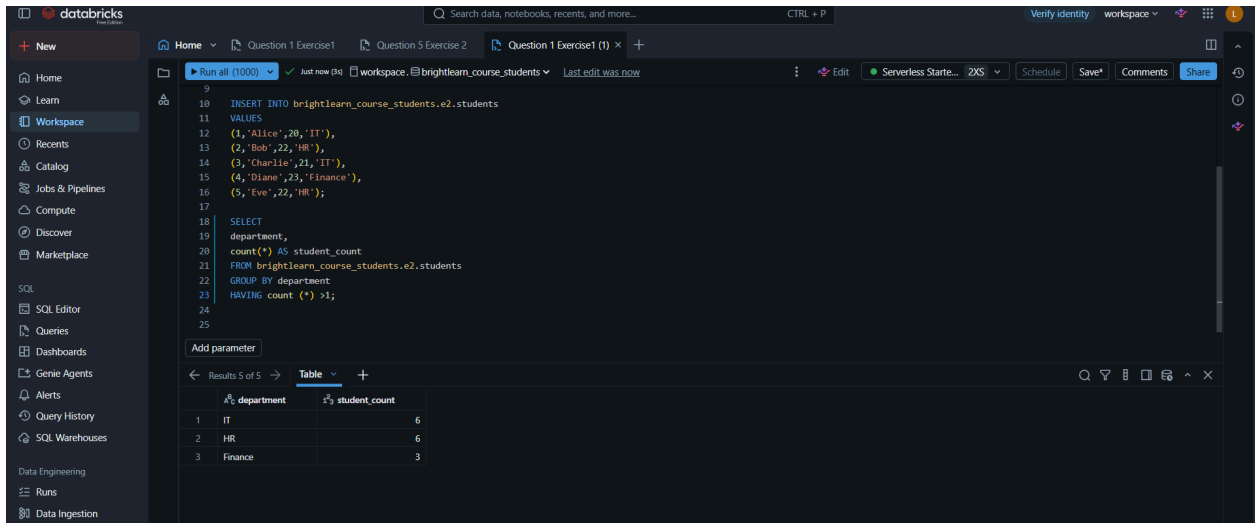
The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.students  
11 VALUES  
12 (1,'Alice',20,'IT'),  
13 (2,'Bob',22,'HR'),  
14 (3,'Charlie',21,'IT'),  
15 (4,'Diane',23,'Finance'),  
16 (5,'Eve',22,'HR');  
17  
18 SELECT  
19 department,  
20 AVG(age) AS avg_age  
21 FROM brightlearn_course_students.e2.students  
22 GROUP BY department;  
23  
24
```

Below the query, the results are displayed in a table format:

department	avg_age
IT	20.5
HR	22
Finance	23

Show departments with more than 1 student.

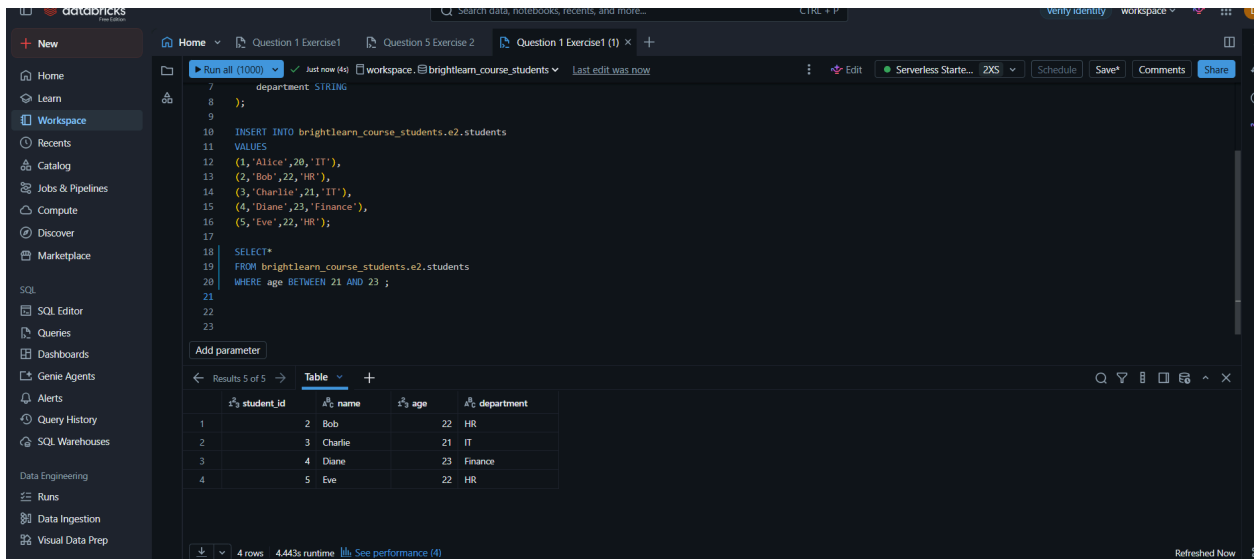


The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays a SQL query with line numbers 9 through 25. The query consists of an INSERT statement followed by a SELECT statement with a HAVING clause. Below the query, the results are displayed in a table format with 5 columns: department, student_count, and student_id. The results show three departments: IT (6 students), HR (6 students), and Finance (3 students).

```
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25
```

department	student_count
IT	6
HR	6
Finance	3

Get all students whose age is between 21 and 23.

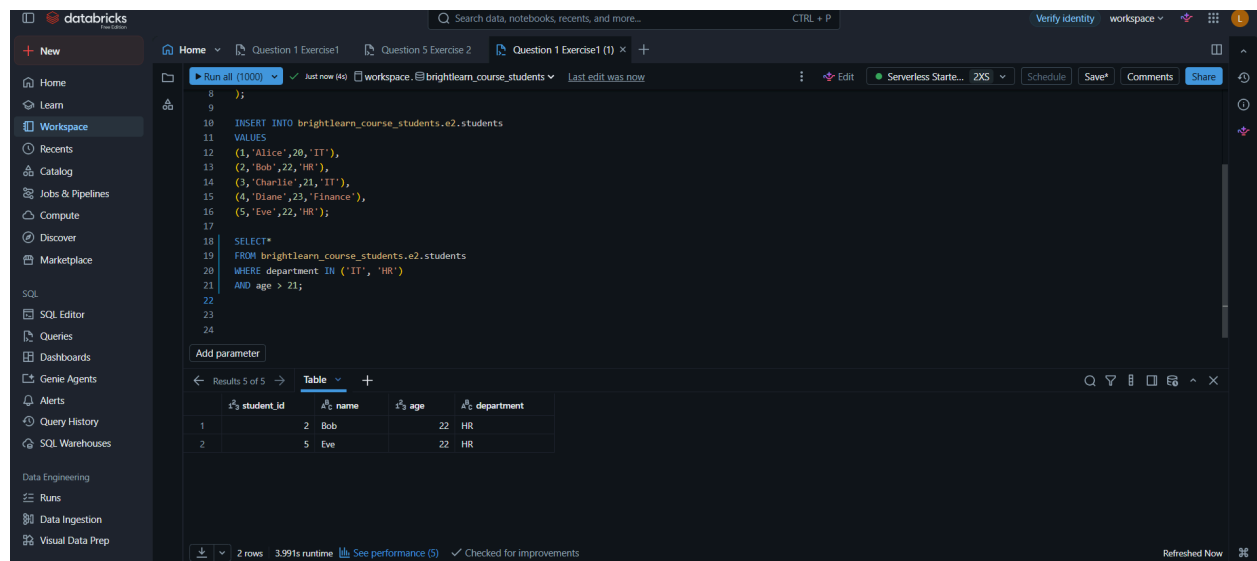


The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays a SQL query with line numbers 7 through 23. The query consists of an INSERT statement followed by a SELECT statement with a WHERE clause. Below the query, the results are displayed in a table format with 5 columns: student_id, name, age, and department. The results show four students: Bob (age 22, HR), Charlie (age 21, IT), Diane (age 23, Finance), and Eve (age 22, HR).

```
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23
```

student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diane	23	Finance
5	Eve	22	HR

List all students in the IT or HR department who are older than 21.



The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

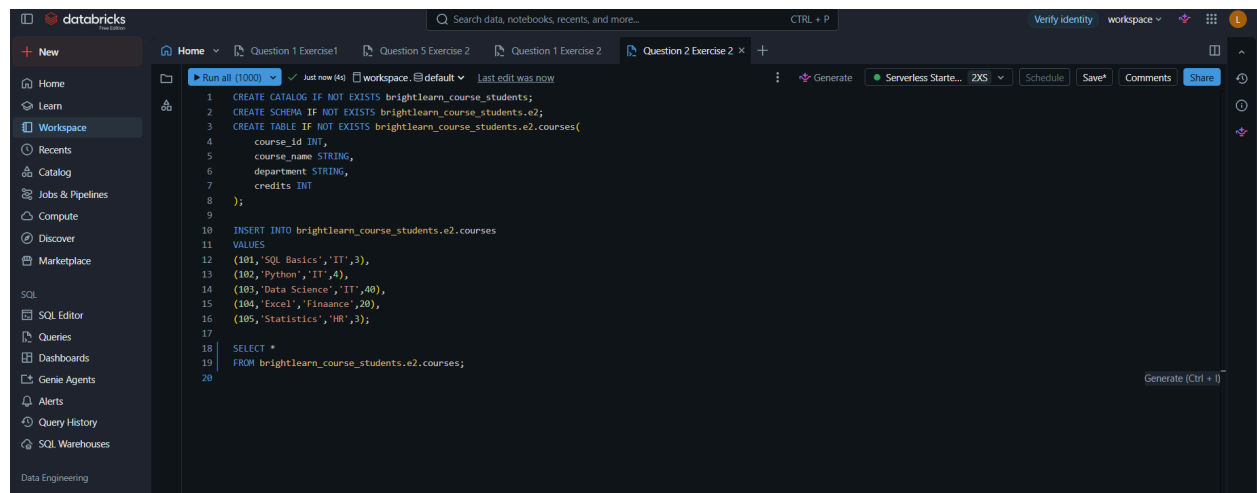
```
8 );
9
10 INSERT INTO brightlearn_course_students.e2.students
11 VALUES
12 (1, 'Alice', 20, 'IT'),
13 (2, 'Bob', 22, 'HR'),
14 (3, 'Charlie', 21, 'IT'),
15 (4, 'Diane', 23, 'Finance'),
16 (5, 'Eve', 22, 'HR');
17
18 SELECT*
19 FROM brightlearn_course_students.e2.students
20 WHERE department IN ('IT', 'HR')
21 AND age > 21;
22
23
24
```

Below the query editor, the results are displayed in a table format. The table has 4 columns: student_id, name, age, and department. The results show 2 rows:

student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

The interface also shows a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The top bar includes a search bar, a workspace dropdown, and a user profile icon.

2.The coursesTable



The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
1 CREATE CATALOG IF NOT EXISTS brightlearn_course_students;
2 CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.e2;
3 CREATE TABLE IF NOT EXISTS brightlearn_course_students.e2.courses(
4   course_id INT,
5   course_name STRING,
6   department STRING,
7   credits INT
8 );
9
10 INSERT INTO brightlearn_course_students.e2.courses
11 VALUES
12 (101, 'SQL Basics', 'IT', 3),
13 (102, 'Python', 'IT', 4),
14 (103, 'Data Science', 'IT', 40),
15 (104, 'Excel', 'Finance', 20),
16 (105, 'Statistics', 'HR', 3);
17
18 SELECT *
19 FROM brightlearn_course_students.e2.courses;
20
```

The interface also shows a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The top bar includes a search bar, a workspace dropdown, and a user profile icon.

Databricks interface showing a SQL query and its results.

```
4 course_id INT,  
5 course_name STRING,  
6 department STRING,  
7 credits INT  
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.courses  
11 VALUES  
12 (101,'SQL Basics','IT',3),  
13 (102,'Python','IT',4),  
14 (103,'Data Science','IT',40),  
15 (104,'Excel','Finance',20),  
16 (105,'Statistics','HR',3);  
17  
18 SELECT *  
19 FROM brightlearn_course_students.e2.courses;  
20
```

Results 5 of 5

	course_id	course_name	department	credits
1	101	SQL Basics	IT	3
2	102	Python	IT	4
3	103	Data Science	IT	40
4	104	Excel	Finance	20
5	105	Statistics	HR	3

Show total credits per department, only for departments with more than 5 total credits

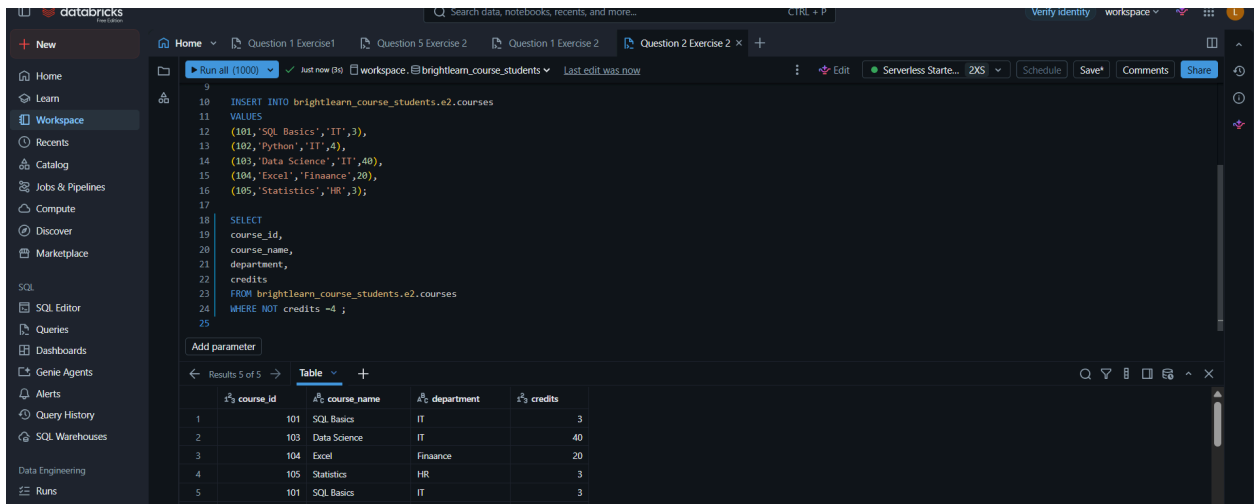
Databricks interface showing a SQL query and its results.

```
7 credits INT  
8 );  
9  
10 INSERT INTO brightlearn_course_students.e2.courses  
11 VALUES  
12 (101,'SQL Basics','IT',3),  
13 (102,'Python','IT',4),  
14 (103,'Data Science','IT',40),  
15 (104,'Excel','Finance',20),  
16 (105,'Statistics','HR',3);  
17  
18 SELECT  
19 department,  
20 SUM(credits) AS total_credits  
21 FROM brightlearn_course_students.e2.courses  
22 GROUP BY department  
23 HAVING SUM(credits) > 5;
```

Results 3 of 5

	department	total_credits
1	IT	141
2	HR	9
3	Finance	60

List all courses that do not have 4 credits.



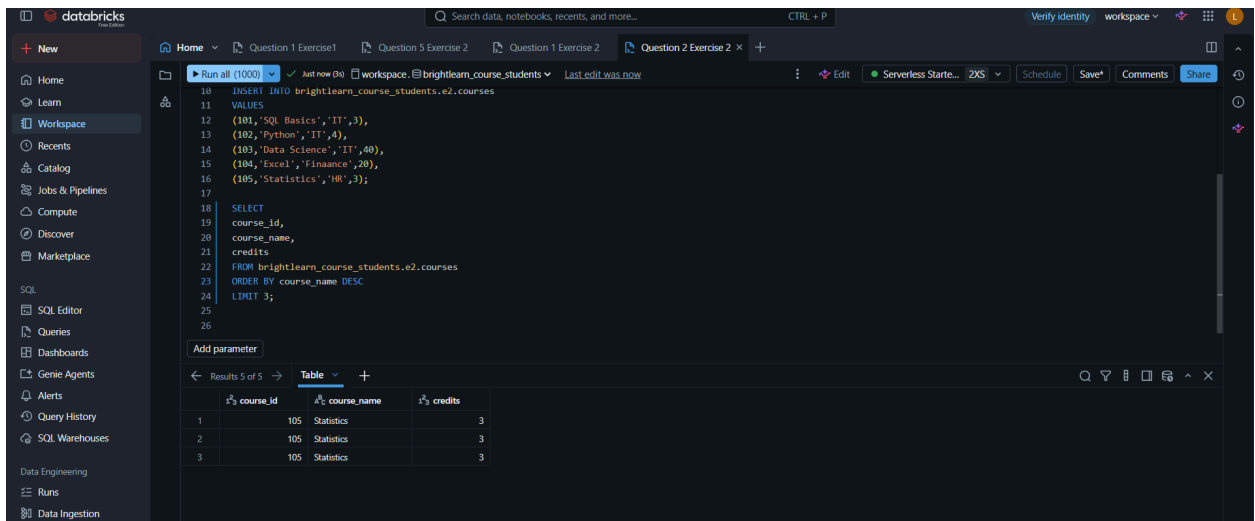
The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
9
10 INSERT INTO brightlearn_course_students.e2.courses
11 VALUES
12 (101,'SQL Basics','IT',3),
13 (102,'Python','IT',4),
14 (103,'Data Science','IT',40),
15 (104,'Excel','Finance',20),
16 (105,'Statistics','HR',3);
17
18 SELECT
19 course_id,
20 course_name,
21 department,
22 credits
23 FROM brightlearn_course_students.e2.courses
24 WHERE NOT credits =4 ;
25
```

Below the query editor, the results are displayed in a table format. The table has 5 columns: `course_id`, `course_name`, `department`, and `credits`. The results show 5 rows of data:

course_id	course_name	department	credits
101	SQL Basics	IT	3
103	Data Science	IT	40
104	Excel	Finance	20
105	Statistics	HR	3
101	SQL Basics	IT	3

Show the top 3 courses by credits in descending order.



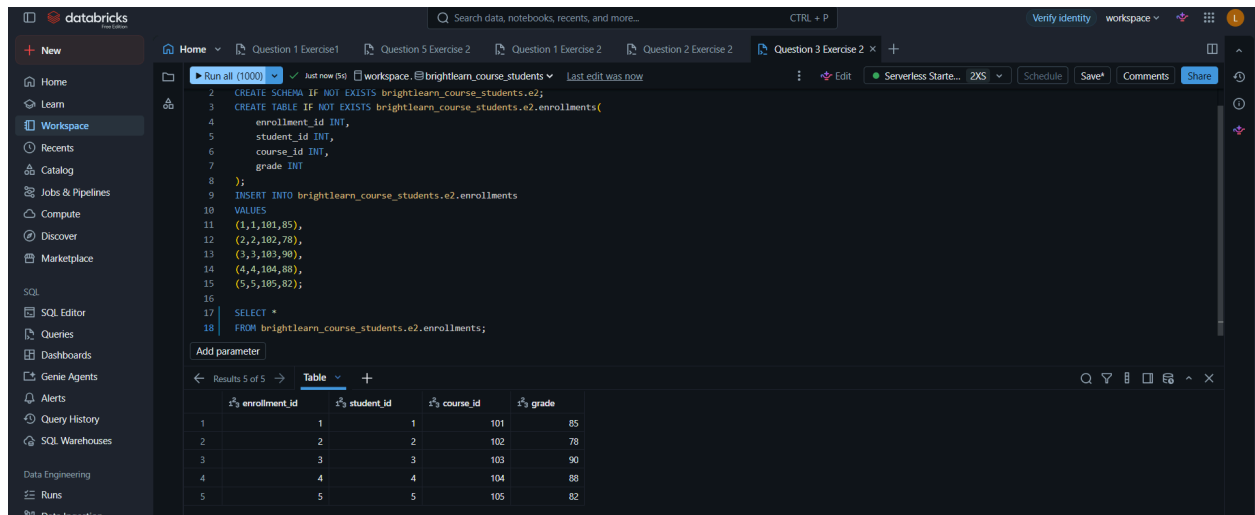
The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
10 INSERT INTO brightlearn_course_students.e2.courses
11 VALUES
12 (101,'SQL Basics','IT',3),
13 (102,'Python','IT',4),
14 (103,'Data Science','IT',40),
15 (104,'Excel','Finance',20),
16 (105,'Statistics','HR',3);
17
18 SELECT
19 course_id,
20 course_name,
21 credits
22 FROM brightlearn_course_students.e2.courses
23 ORDER BY course_name DESC
24 LIMIT 3;
25
26
```

Below the query editor, the results are displayed in a table format. The table has 3 columns: `course_id`, `course_name`, and `credits`. The results show 3 rows of data, ordered by `course_name` in descending order:

course_id	course_name	credits
105	Statistics	3
105	Statistics	3
105	Statistics	3

3.The enrollments Table

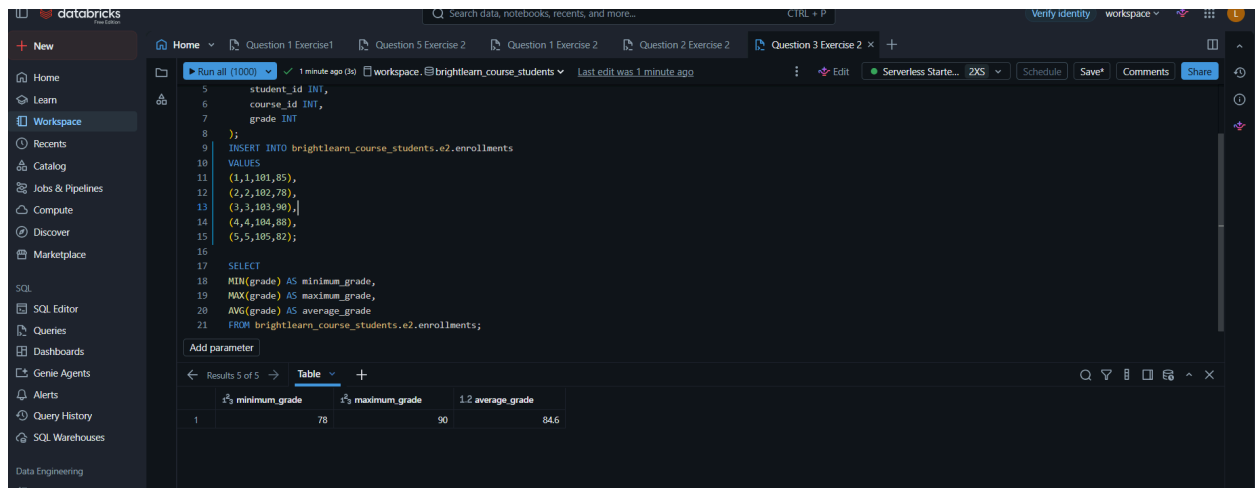


The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays a SQL script that creates a schema, a table named 'enrollments', and inserts five rows of data. The table has columns: enrollment_id, student_id, course_id, and grade. Below the script, the results of the execution are shown in a table format.

```
2 CREATE SCHEMA IF NOT EXISTS brightlearn_course_students;
3 CREATE TABLE IF NOT EXISTS brightlearn_course_students.e2.enrollments(
4   enrollment_id INT,
5   student_id INT,
6   course_id INT,
7   grade INT
8 );
9 INSERT INTO brightlearn_course_students.e2.enrollments
10 VALUES
11 (1,1,101,85),
12 (2,2,102,78),
13 (3,3,103,90),
14 (4,4,104,88),
15 (5,5,105,82);
16
17 SELECT *
18 FROM brightlearn_course_students.e2.enrollments;
```

enrollment_id	student_id	course_id	grade
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

Get the maximum, minimum, and average grade across all enrollments.



The screenshot shows the Databricks SQL Editor interface with a new SQL query. The query calculates the minimum, maximum, and average grade across all enrollments. The results are displayed in a table format.

```
5   student_id INT,
6   course_id INT,
7   grade INT
8 );
9 INSERT INTO brightlearn_course_students.e2.enrollments
10 VALUES
11 (1,1,101,85),
12 (2,2,102,78),
13 (3,3,103,90),
14 (4,4,104,88),
15 (5,5,105,82);
16
17 SELECT
18 MIN(grade) AS minimum_grade,
19 MAX(grade) AS maximum_grade,
20 AVG(grade) AS average_grade
21 FROM brightlearn_course_students.e2.enrollments;
```

minimum_grade	maximum_grade	average_grade
78	90	84.6

Count how many enrollments exist per course.

The screenshot shows the Databricks SQL Editor interface. The query being executed is:

```
5 student_id INT,  
6 course_id INT,  
7 grade INT  
8 );  
9 INSERT INTO brightlearn_course_students.e2.enrollments  
10 VALUES  
11 (1,1,101,85),  
12 (2,2,102,78),  
13 (3,3,103,90),  
14 (4,4,104,88),  
15 (5,5,105,82);  
16  
17 SELECT  
18 course_id,  
19 COUNT(*) AS enrollment_count  
20 FROM brightlearn_course_students.e2.enrollments  
21 GROUP BY course_id;
```

The results of the query are displayed in a table with 5 rows and 2 columns:

course_id	enrollment_count
105	13
104	13
103	13
102	13
101	13

4.The salariesTable

The screenshot shows the Databricks SQL Editor interface. The query being executed is:

```
5 name STRING,  
6 department STRING,  
7 salary INT,  
8 bonus INT  
9 );  
10  
11 INSERT INTO brightlearn_course_students.e2.salaries  
12 VALUES  
13 (1,'Tom','IT',60000,5000),  
14 (2,'Jerry','HR',55000,4000),  
15 (3,'Spike','Finance',70000,6000),  
16 (4,'Tyke','IT',62000,5500),  
17 (5,'Butch','HR',54000,3500);  
18  
19 SELECT*  
20 FROM brightlearn_course_students.e2.salaries;  
21
```

The results of the query are displayed in a table with 5 rows and 5 columns:

employee_id	name	department	salary	bonus
1	Tom	IT	60000	5000
2	Jerry	HR	55000	4000
3	Spike	Finance	70000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

Find total salary and total bonus per department.

The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
9 );
10
11 INSERT INTO brightlearn_course_students.e2.salaries
12 VALUES
13 (1,'Tom','IT',60000,5000),
14 (2,'Jerry','HR',55000,4000),
15 (3,'Spike','Finance',70000,6000),
16 (4,'Tyke','IT',62000,5500),
17 (5,'Butch','HR',54000,3500);
18
19 SELECT
20 department,
21 SUM(salary) AS total_salary,
22 SUM(bonus) AS total_bonus
23 FROM brightlearn_course_students.e2.salaries
24 GROUP BY department;
```

The results table shows the following data:

department	total_salary	total_bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

Show departments where average salary is above 55,000.

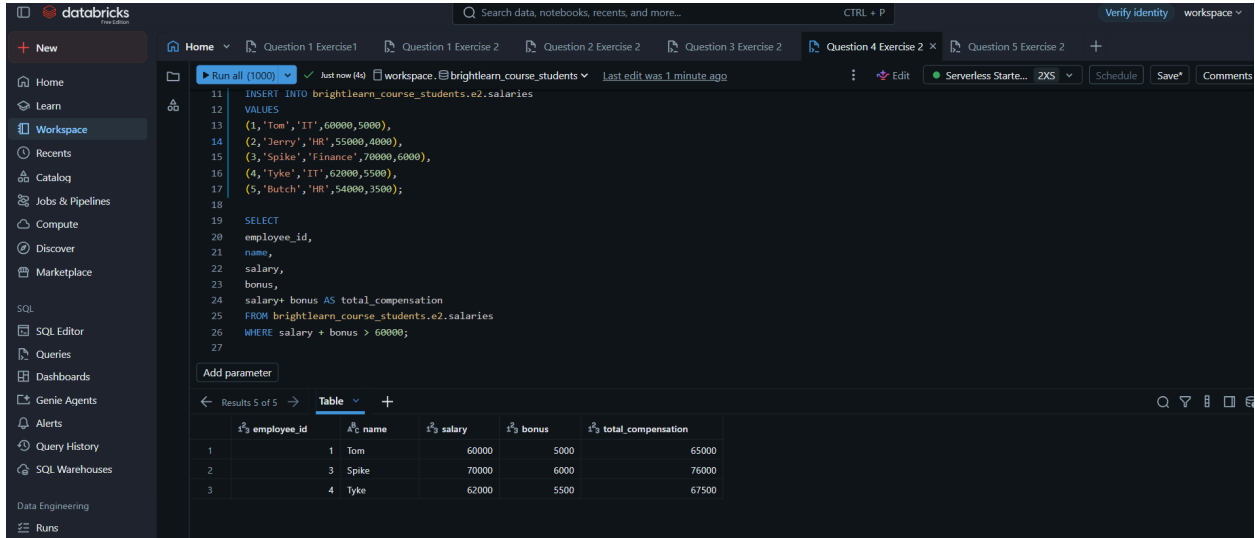
The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
10 );
11 INSERT INTO brightlearn_course_students.e2.salaries
12 VALUES
13 (1,'Tom','IT',60000,5000),
14 (2,'Jerry','HR',55000,4000),
15 (3,'Spike','Finance',70000,6000),
16 (4,'Tyke','IT',62000,5500),
17 (5,'Butch','HR',54000,3500);
18
19 SELECT
20 department,
21 AVG(salary) AS average_salary
22 FROM brightlearn_course_students.e2.salaries
23 GROUP BY department
24 HAVING AVG(salary)>55000;
```

The results table shows the following data:

department	average_salary
IT	61000
Finance	70000

List employees whose salary plus bonus is greater than 60,000



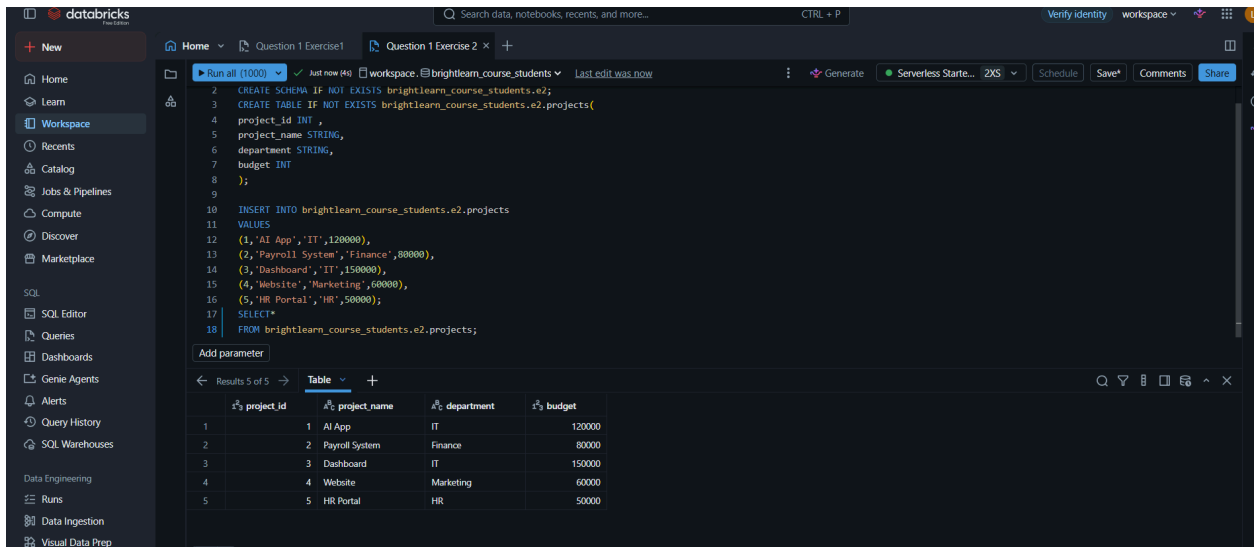
The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
11 INSERT INTO brightlearn_course_students.e2.salaries
12 VALUES
13 (1,'Tom','IT',60000,5000),
14 (2,'Jerry','HR',55000,4000),
15 (3,'Spike','Finance',70000,6000),
16 (4,'Tyke','IT',62000,5500),
17 (5,'Butch','HR',54000,3500);
18
19 SELECT
20 employee_id,
21 name,
22 salary,
23 bonus,
24 salary+ bonus AS total_compensation
25 FROM brightlearn_course_students.e2.salaries
26 WHERE salary + bonus > 60000;
27
```

Below the query editor, the results are displayed in a table format:

employee_id	name	salary	bonus	total_compensation
1	Tom	60000	5000	65000
2	Spike	70000	6000	76000
3	Tyke	62000	5500	67500

5.The Projects Table



The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
2 CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.e2;
3 CREATE TABLE IF NOT EXISTS brightlearn_course_students.e2.projects(
4 project_id INT,
5 project_name STRING,
6 department STRING,
7 budget INT
8 );
9
10 INSERT INTO brightlearn_course_students.e2.projects
11 VALUES
12 (1,'AI App','IT',120000),
13 (2,'Payroll System','Finance',80000),
14 (3,'Dashboard','IT',150000),
15 (4,'Website','Marketing',60000),
16 (5,'HR Portal','HR',50000);
17 SELECT*
18 FROM brightlearn_course_students.e2.projects;
```

Below the query editor, the results are displayed in a table format:

project_id	project_name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
3	Dashboard	IT	150000
4	Website	Marketing	60000
5	HR Portal	HR	50000

Show total and average budget per department. Only include departments with average budget above 70,000.

The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
9
10 INSERT INTO brightlearn_course_students.e2.projects
11 VALUES
12 (1,'AI App','IT',120000),
13 (2,'Payroll System','Finance',80000),
14 (3,'Dashboard','IT',150000),
15 (4,'Website','Marketing',60000),
16 (5,'HR Portal','HR',50000);
17
18 SELECT
19 department,
20 SUM(budget) AS total_budget,
21 AVG(budget) AS avg_budget
22 FROM brightlearn_course_students.e2.projects
23 GROUP BY department
24 HAVING AVG (budget)> 70000;
25
```

The results table shows the following data:

department	total_budget	avg_budget
IT	540000	135000
Finance	160000	80000

List all projects with budgets between 50,000 and 120,000, excluding the Marketing department.

The screenshot shows the Databricks SQL Editor interface. The query in the editor is as follows:

```
10 INSERT INTO brightlearn_course_students.e2.projects
11 VALUES
12 (1,'AI App','IT',120000),
13 (2,'Payroll System','Finance',80000),
14 (3,'Dashboard','IT',150000),
15 (4,'Website','Marketing',60000),
16 (5,'HR Portal','HR',50000);
17
18 SELECT
19 project_id,
20 project_name,
21 department,
22 budget
23 FROM brightlearn_course_students.e2.projects
24 WHERE budget BETWEEN 50000 AND 120000
25 AND department NOT IN ('Marketing');
26
```

The results table shows the following data:

project_id	project_name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
3	HR Portal	HR	50000